SAS/R Monitoring & On-Leave Coverage

Turn your already-validated monitoring programs into a **scheduled, self-escalating loop** so the trial keeps being watched, the tracker keeps being current, and nothing falls through the cracks — **especially while you're out on leave**. The monitoring itself is **deterministic SAS/R, no AI required** — the lowest-risk, ship-today pipeline in this whole program.

Three components, one scheduled job. The deterministic **TRIALMON** backbone does the watching; **SHEETLINK** keeps the tracker current and **LOCALMIND** is an optional language convenience — every path routes to humans, who decide.

The one rule. These macros **detect, package, and email pre-specified deterministic flags**. They **do not triage, interpret, or adjudicate**, and they **never auto-act on safety**. A flag is a *prompt for a human* — the medical monitor and the covering biostatistician make every call. Validate the threshold logic + scheduling wrapper as study programs (GxP); keep anything *reported* independently double-programmed; reported numbers come from validated tools, never from a flag and never from the optional language model.

Three components, one scheduled job

TRIALMON · the backbone

Deterministic SAS/R that ingests the latest data, gates on *freshness*, runs the pre-specified safety + operational *checks*, rolls up severity, emails a *digest*, fires a *tier-2 urgent alert* on any RED, and closes with a *heartbeat* — watched by an independent *dead-man's switch*. The `%tm_*` library.

SHEETLINK · the tracker

The same scheduled job keeps your Smartsheet program tracker *current* — idempotent upsert-by-key behind an ops-only allowlist guard — so the PM dashboard is live while you're away. The `%ss_*` library. No AI.

LOCALMIND · optional language

An *optional* on-device SLM drafts the digest prose or triages free-text the deterministic code can't — ops-only, behind the loopback + validator + human-gate guardrails. The monitoring needs **none** of this; it is a convenience, not a dependency. The `%slm_*` library.

Why this is the lowest-risk piece. The detection is pre-specified, deterministic SAS/R — the same logic you already validate, just scheduled and self-escalating. No cloud, no AI in the loop, no model to qualify. It *detects,*

packages, and pages; humans decide. That is exactly the property a regulated shop wants from unattended automation.

Below: the architecture, the checks, the alerting + escalation, the covering-while-on-leave runbook (the headline), the tracker, the optional language layer, the macro libraries, the safeguards, and IT readiness.

Watch it work — written in SAS, then a month of coverage

First, the monitoring backbone *written line by line and run* in SAS Enterprise Guide — in the Azure session where the study data actually lives. Then the on-leave scenario end to end: the daily job runs under a service account, a RED finding pages the covering biostatistician with an evidence packet, the tracker stays current, and the dead-man's switch guarantees you'd know if it ever went quiet.

▶ The monitor, written & run in SAS — live screencast **▶ Covering while on leave — live screencast**

⤓ Picture guide (PDF)

And the two companions in action:

▶ Smartsheet keeps itself current — live screencast **▶ On-device language triage (optional)**

Every section, demonstrated on screen

Each part of this wiki has its own live screencast — the actual SAS (or R) code written line by line in SAS Enterprise Guide 8.3 on the Azure desktop, and every icon clicked. No slides.

▶ Architecture

The daily job wired in `tm_config.sas` — libraries, thresholds, recipients, schedule.

▶ The %tm_* library

Inside `tm_macros.sas`: the freshness gate, the roll-up, written line by line.

▶ The safety checks

The Hy's-Law AND-gate discipline and the CTCAE labs band — thresholds, not diagnoses.

▶ Alerts & escalation

The escalation matrix written, then the tier-2 route shown in the log.

▶ The R companion

Prototyping in RStudio on the laptop — synthetic, aggregate, never study data.

▶ The optional language layer

The on-device SLM boxed in: loopback, a string-only schema, a human gate.

▶ IT readiness

Scheduling the job under a service account in Windows Task Scheduler + the watchdog.

The scenario: you are out for three weeks. Each morning the job ingests the overnight EDC/lab/ECG exports, checks freshness first, runs the safety flags, and emails a digest to the support alias. On day 9 an eDISH point trips Hy's-Law screening — a tier-2 alert pages the covering biostatistician with a one-page evidence packet;

they loop in the medical monitor, who makes the call. The tracker reflects it within the hour. You return to a complete trail, not a backlog.

The interactive dashboard — the backbone made visible

A self-contained, click-through dashboard on a **synthetic, longitudinal** CP-101 run (22 participants × 8 visits), presented as a **dark operations console** (status pill, a what-if *scenario simulator*, cohort *ranking bars*, a freshness strip, and a network view). It is a *teaching view* of what the deterministic loop produces — not a production console, not a record. Open it in any browser; nothing leaves the file.

▸ **Open the TRIALMON dashboard** | 📖 **The Field Guide (how to read it)**

What it is and isn't. It runs on **synthetic data** — no PHI, no real participants, no unblinding. Every flag is a *screening prompt*, never a determination and never a reportable number; reported numbers (incidence tables, the ICH E14 table, central reads, MTD, final disposition) come only from validated tools (Phoenix WinNonlin, Pinnacle 21). It is an *illustration* of the backbone, not the validated job itself.

One tab is fenced off as exploratory. An *"Exploratory ▸ structure"* section adds unsupervised **persistence-homology** trial-structure analytics — H_0 clustering of participants in safety-feature space (the β_0 curve + barcode + a PCA embedding) and a site network — for hypothesis generation only. It is **explicitly NOT part of the validated safety backbone**: it never produces a flag, a determination, or a number, and nothing there escalates study status. The deterministic-flag participants happen to surface as topological outliers there too — an independent structural cross-check, not a second opinion that decides anything.

What it shows — seven validated sections + one exploratory, per-visit detail

A section sub-nav (Overview · Hepatic · Cardiac · Labs & Vitals · AE & DLT · Disposition · Participants) with a KPI tile strip and a standing governance ribbon. Every per-visit series is built so its worst on-treatment value equals the original summary — nothing contradicts the headline plots.

Seven validated sections, one fenced-off tab. Everything inside the green zone is the governed teaching view of the deterministic backbone; the **exploratory** persistence-homology tab sits deliberately outside it and never produces a flag, number, or status change.

- **Overview** — KPI tiles; a prioritized *attention worklist* (every open AMBER/RED screening flag, click-to-open); a first-class *data-freshness/SLA* panel (stale-green → RED); and a *study heatmap* (participant × visit, worst flag — the "when did it start" view).
- **Hepatic** — the eDISH scatter now with per-visit *history trails* (where a participant has been, not a prediction); per-visit *LFT trajectories*; an *R-ratio pattern* strip; and a peak/baseline/trend table. The near-miss (high ALT, bilirubin under 2×) is correctly kept GREEN by the AND-gate at every visit.
- **Cardiac** — per-visit QTcF (Fridericia) trajectory with a cohort-median band, and an ICH E14 categorical outlier *census* (clearly labelled "screening census — not the E14 table"; baseline-watch shown distinctly).
- **Labs & Vitals** — a baseline→worst *lab-shift grid* (CTCAE-style screening bands, not adjudicated grades), hematology trajectories, a renal fold-rise panel (eGFR an estimate), and a vitals/orthostatic PCS board (operational context).
- **AE & DLT** — a 3+3 escalation *state ladder* (trigger / decision-pending only — the SRC decides, never the dashboard), an AE SOC×CTCAE-grade matrix (severity and seriousness kept separate), and an AE onset swimlane vs study day.
- **Disposition** — an enrollment/disposition funnel, RBQM KRI tiles + a protocol-deviation log, and a visit-compliance matrix (a missed visit is routed to a human, never read as "fine").
- **Participants** — a sortable/filterable/searchable roster; and on **click anywhere**, an evidence-packet modal with per-visit sparkline trajectories.

A full field guide ships with it. The dashboard comes with a dedicated, operating-depth Field Guide — a 15-section reference that documents every section, panel, threshold and governance rule, the synthetic data model, and a guided tour of the planted teaching cases. Read it next to the dashboard the first time through.

Architecture — the daily job

One orchestrated batch job (`monitor_driver.sas`), scheduled under a **service account**, runs the loop. Order matters: **freshness gates everything**, and every fragile step is wrapped to **fail loud**.

▶ **Watch it wired in config (tm_config.sas) — live screencast**

1. Ingest — resolve the newest dated exports (`%tm_latest`)
↓
2. Freshness gate — is each feed fresh? **a dead feed flags loudest; stale-green is worse than red** (`%tm_freshness`)
↓
3. Guard — assert expected columns exist, are numeric, pass a range-sanity check (`%tm_guard`)
↓
4. Checks — the pre-specified deterministic safety + operational flags (`%tm_chk_*`)
↓
5. Roll-up — worst severity GREEN<AMBER<RED (`%tm_status`)
↓
6. Digest — email the daily summary to the support alias (`%tm_digest`)
↓
7. Tier-2 alert — on any *de-duplicated* RED, page with an evidence packet (`%tm_alert`)
↓
8. Heartbeat — write a run record, always, even on failure (`%tm_heartbeat`)

The independent dead-man's switch. `tm_watchdog.sh` runs on a *different host and account*, reads the heartbeat, and screams if the job didn't run, ran late, or ran on stale data. The thing most likely to fail silently is the scheduler itself — so the watchdog is the control that makes "no news" trustworthy. **No email is itself a signal.**

The two branches off the same job

- **Tracker (SHEETLINK):** after the checks, `%ss_*` upserts the run status + open flags into the Smartsheet tracker — idempotent by key, ops-only columns allowlisted — so the PM view is live. See Keeping the tracker current.
- **Language (LOCALMIND, optional):** if enabled, an on-device SLM drafts the digest prose from the already-structured rows — never deciding what to include. See The optional language layer.

The monitoring checks — deterministic flags

Each check reads validated analysis data and writes a flagged dataset with a `sev` column (GREEN / AMBER / RED) against **pre-specified thresholds**. They are **screening flags, not diagnoses** — a RED is a prompt for a human, not a finding.

▶ **Watch the AND-gate discipline & the CTCAE band — live screencast**

Check	What it flags	Default thresholds
`%tm_chk_hyslaw`	eDISH / Hy's-Law screening: ALT or AST >×ULN **and** TBili >×ULN	ALT/AST 3×, TBili 2×, ALP 2× — *screening flag, not a diagnosis*
`%tm_chk_qtcf`	QTcF absolute & change-from-baseline tiers (ICH E14)	RED abs 500 / Δ60; AMBER abs 480 / Δ30
`%tm_chk_ae`	AE/SAE running tally by cohort — participant-incidence + event counts	study-specified
`%tm_chk_dlt`	Candidate-DLT tally vs the 3+3 rule (against an **adjudicated** DLT flag + completed window)	*does not make the escalation decision*
`%tm_chk_labs`	Out-of-range / potentially-clinically-significant / shift flags	study-specified

Five deterministic checks, one rolled-up status. Each **%tm_chk_*** reads validated data and writes a `sev` column against pre-specified thresholds; **%tm_status** takes the worst severity — a RED is a screening prompt for a human, never a diagnosis.

Two disciplines that make unattended checks safe. Fail loud — `%tm_assert` wraps every fragile step; on a false condition it sets the return code, forces status to ERROR, emails the backup, and writes a FAILED heartbeat, so a broken run is never a silent green. **Guard the inputs** — `%tm_guard` asserts the expected columns exist, are numeric, and pass a range-sanity check before any threshold runs, catching a unit change or a renamed variable that would otherwise sail through. And the **freshness gate runs first**: a missing or dead feed flags *loudest*, because **stale-green is the most dangerous state** — it looks fine and isn't.

Alerting, paging & escalation

Two tiers, de-duplicated so a standing RED pages *once*, plus a heartbeat that makes silence meaningful.

▶ **Watch the escalation matrix written & routed — live screencast**

- **Tier 1 — the daily digest** (`%tm_digest`): every run emails a summary to the support alias, RED/AMBER/GREEN by section. Routine awareness.
- **Tier 2 — the urgent alert** (`%tm_alert`): on any *new* de-duplicated RED, page the support alias + a named backup with a one-page **evidence packet** (`%tm_evidence` — the eDISH scatter with reference lines + the participant's lab trajectory) and the medical monitor's name + phone. Enough to act, not to decide.
- **The heartbeat** (`%tm_heartbeat`): a run record written *always*, success or failure — the input to the watchdog.

Two tiers plus a heartbeat that makes silence mean something. Tier 1 is routine awareness; tier 2 pages a named backup with an evidence packet on a new RED; the always-written heartbeat feeds an independent watchdog so **no email is itself a signal**.

Escalation matrix

Colour	Means	Who acts
GREEN	Checks ran, fresh data, nothing tripped	No action — the digest is the record
AMBER	A tier threshold crossed, or a feed is ageing	Covering biostatistician reviews same day
RED	A safety screen tripped, or a feed is dead/stale	Page → covering biostatistician loops in the medical monitor, who makes the call
NO EMAIL	The job didn't run (the worst case)	The watchdog pages the backup — treat as RED until proven otherwise

> **Reportable vs signal-only.** A flag is an internal *signal* to look — it is not a reportable safety determination. Reportability (expedited SAE, DSUR, etc.) is the medical monitor's and sponsor's decision through the validated process; the automation accelerates *awareness*, never the regulatory call.

Covering while out on leave — the runbook

The headline use case. Hand the covering biostatistician one page: what runs and when, what the colours mean, who they call, and what to do if the email ever stops. Everything runs under the service account `SVC-BIOSTAT`, never a personal login — so it does not break the day you turn off your laptop.

> **What runs, and when.** The daily monitoring job at a fixed time (e.g. 06:00, weekdays) under `SVC-BIOSTAT`; the watchdog ~30 min later on a *different* host; the tracker upsert in the same job. The covering biostatistician is the named `backup` on every tier-2 alert and every watchdog page for the leave window — one line in `tm_config.sas`, not a code change.

A worked three-week leave

1. **Before you go:** set `backup=` to the covering biostatistician and confirm the medical monitor's contact in `tm_config.sas`; send them this runbook + the picture guide; run the job once and confirm the digest, a test alert, and a heartbeat all arrive.
2. **Days 1–8:** GREEN digests each morning; the tracker stays current; the covering biostatistician does nothing but glance.
3. **Day 9:** an eDISH point trips Hy's-Law screening — a tier-2 alert pages the covering biostatistician with the evidence packet; they review, loop in the medical monitor, who makes the call; the tracker reflects the open flag within the hour.
4. **Day 14:** an overnight feed fails to land — the freshness gate flags it RED and the watchdog confirms the job ran but on a dead feed; IT restores the export; no silent gap.
5. **On return:** a complete, dated trail of digests, the one alert, the resolution, and the tracker history — not a backlog to reconstruct.

> **Accountability does not go on leave.** The automation covers *vigilance and logistics* — it never covers the *decision*. A named covering biostatistician and the medical monitor own every call while you are out; the runbook makes the hand-off explicit so there is never an ambiguous "who's watching" gap. Tune thresholds before leave to avoid alert fatigue, but never widen a safety threshold to silence a page.

⤓ **The Coverage Runbook (PDF) — hand this to your cover**

Keeping the Smartsheet tracker current — SHEETLINK

So the PM's program dashboard is live while you're away, the same scheduled job upserts status into Smartsheet — deterministically, no AI.

- **Idempotent upsert-by-key:** `%ss_*` matches each row by a stable key and updates-or-inserts, so re-runs never duplicate and a re-processed day is a no-op.
- **Ops-only allowlist guard:** only an allowlisted set of operational columns can be written — the automation can never touch a column it shouldn't, by construction.
- **Token from a secret, never logged:** the Smartsheet token comes from an environment variable / secret store, never hard-coded and never written to a log.

> Independently SAS-reviewed (three defects found and fixed). The tracker shows *operational* status only — never a reported number, never patient-level data.

Open the SHEETLINK wiki | Macro README (PDF)

The optional on-device language layer — LOCALMIND

Optional, and ops-only. The monitoring needs **no AI** — the checks, the digest structure, the tracker, and every number are deterministic SAS/R. The on-device SLM is a *convenience* for the language tasks code can't do (re-phrasing a digest, triaging a free-text comment), and it runs behind the same guardrails as the rest of the program: an **on-prem frozen model on loopback**, schema-constrained output, an allowlist validator, and a **human-gated parse-guard**. It never produces a number and never decides what the digest includes.

If you enable it, the `%slm_*` library (loopback-guard `%slm_init`, schema-constrained `%slm_chat`, allowlist `%slm_validate`, `%slm_classify`) is the engine, and the standard honesty applies: a small local model hallucinates, so its trustworthiness comes from the discipline around it, not from being local. See the dedicated on-device wiki for the full treatment.

▶ **Watch it boxed in (loopback + string-only schema + human gate) — live screencast**

On-device SLM × SAS/R wiki | **IT enablement runbook**

The macro libraries

Three deterministic SAS libraries (each with an R companion), copy-paste ready, each with a per-study config so a second study or a rotating monitor is one file, not a code edit. Base SAS 9.4; no third-party packages.

▶ Watch inside the %tm_* library — live screencast

▶ The R companion on the laptop (RStudio)

Library	Driver + key files	What it does
TRIALMON `%tm_*`	`tm_macros.sas` · `monitor_driver.sas` · `tm_config.sas` · `tm_watchdog.sh` · `tm_companion.R`	The monitoring loop: init, latest, freshness, guard, the safety checks, status, digest, alert, evidence, heartbeat — plus the independent watchdog.
SHEETLINK `%ss_*`	`ss_macros.sas` · `tracker_update.sas` · `ss_config.sas` · `ss_companion.R`	Idempotent upsert-by-key into Smartsheet behind an ops-only allowlist guard; token from a secret.
LOCALMIND `%slm_*`	`slm_macros.sas` · `triage_driver.sas` · `slm_config.sas` · `slm_companion.R`	(Optional) the on-device language layer: loopback guard, schema-constrained call, allowlist validator, classify.

Config, not code. `tm_config.sas` holds the per-study constants — paths, recipients, the medical monitor, feeds, thresholds — `%include`d before the driver. A new study, a rotating medical monitor, or a leave-coverage backup is a one-line change, change-controlled like any study program.

TRIALMON README

SHEETLINK README

LOCALMIND README

Safeguards & governance

> **Detect, package, page — never decide.** The automation surfaces pre-specified deterministic flags and routes them to people. It does **not** triage, interpret, adjudicate, or auto-act on safety. The medical monitor and the covering biostatistician make every call; reportability is the validated regulatory process, not a flag.

The controls that make unattended automation defensible

- **Deterministic, validated logic.** The checks are the same threshold logic you already validate — just scheduled. Validate the threshold programs *and* the scheduling wrapper as study programs (GxP, risk-based CSV); version-control the config and any threshold change.
- **Fail loud + guard the inputs.** `%tm_assert` turns any broken step into a loud ERROR + a backup email + a FAILED heartbeat; `%tm_guard` catches a unit change or renamed variable before a threshold runs. A broken run is never a silent green.
- **Freshness first.** Stale-green is the most dangerous state; a dead feed flags loudest.
- **The independent watchdog.** A dead-man's switch on a different host/account makes "no email" a signal, not a guess.
- **Service account, not a person.** The job runs under `SVC-BIOSTAT` so coverage doesn't depend on anyone's laptop; the backup is named per leave window.
- **Numbers stay validated.** Anything *reported* is independently double-programmed and comes from validated tools (Phoenix WinNonlin, Pinnacle 21, the validated pipeline). A flag is never a reported value; the optional SLM never produces one.

IT readiness — the lowest-friction path in the program

For the deterministic monitoring + tracker, the asks are small and entirely on-prem — no cloud, no AI to qualify. Hand IT this list.

▶ **Watch it scheduled under a service account (Task Scheduler) — live screencast**

Deterministic core — ship today
entirely on-prem · no cloud · no AI to qualify

- ✓ Service account SVC-BIOSTAT (no PTO orphaning)
- ✓ Batch schedule + a 2nd host for the watchdog
- ✓ Internal SMTP relay (no external mail)
- ✓ Read access to exports + a state directory
- ✓ Smartsheet token in a secret store (tracker)

no licences beyond what you have

Optional language layer only
heavier asks — keep this request separate

- \+ On-prem workstation / VM
- \+ Signed runtime + a frozen open-weight model
- \+ Licence review
- \+ GAMP-5 qualification of a non-deterministic part

the core needs NONE of this
don't let the AI ask hold up the low-risk piece

Two separate requests. The deterministic monitoring + tracker core is small, on-prem, and has nothing to qualify; only the **optional** language layer adds a model to validate — keep the asks separate so the low-risk piece ships now.

What to ask IT for (deterministic core):

- A **service / group account** (`SVC-BIOSTAT`) to own the scheduled job and read the analysis libraries — so the automation never orphans on PTO.
- **Batch scheduling** (Windows Task Scheduler / cron) for the daily job, and a **second host/account** for the watchdog dead-man's switch.
- An **internal SMTP relay** for the digest + alerts (no external mail).
- Read access to the data exports + a state directory for de-duplication and heartbeats.
- For the tracker: a **Smartsheet API token in a secret store** (never hard-coded).

Only if you enable the optional language layer do the heavier asks apply — an on-prem workstation/VM, a signed runtime + a frozen open-weight model, a licence review, and the GAMP-5 qualification of a non-deterministic component (see the SLM enablement runbook). The monitoring core needs none of it; keep the two requests separate so the low-risk piece isn't held up by the AI one.

Maintenance & FAQ

Monthly upkeep (~15 min)

- Skim the heartbeat log for missed/late runs; confirm the watchdog fired in any test.
- Review alert volume — persistent AMBER noise means a threshold needs a (change-controlled) review, not silencing.
- Confirm `tm_config.sas` recipients + the medical monitor + the current leave backup are correct.

FAQ

Does it make safety decisions? No. It detects, packages, and pages pre-specified deterministic flags; humans decide. It never auto-acts on safety and never adjudicates.

Is there AI in the loop? Not in the monitoring — it is deterministic SAS/R. The on-device SLM is an optional, ops-only language convenience behind guardrails; the numbers and the flags never depend on it.

What if the job silently dies? The independent watchdog pages the backup — *no email is itself a signal*. That is the whole point of the heartbeat + dead-man's switch.

Does it cost anything? The deterministic core is just scheduled SAS/R on a service account + internal SMTP — no licences beyond what you have. Only the optional SLM adds a real (validation + ownership) cost.

Is it validated? Validate the threshold logic + the scheduling wrapper as study programs, risk-based; version-control thresholds; double-program anything reported. The automation is decision-support and a logistics aid, not a record generator.

How does coverage actually work while I'm out? See the coverage runbook — service-account scheduling, a named backup per leave window, the escalation matrix, and the dead-man's switch, all on one page you hand to your cover.